Madár Patrik

Projekt dokumentáció

RAG, LLM, VPS

2026. október

Contents

1 RAG assistant **3**
1.1 RAG modell 3
1.2 Implementáció 3

Chapter 1

RAG assistant

1.1 RAG modell

Szeretnénk olyan AI asszisztenst alkotni, ami képes a saját tudástárát ötvözve megadott dokumentumokat figyelembe véve választ adni a felhasználónak.
Az alapmodell tehát egy vagy több kérdéssel és válaszoknak egy halmazával dolgozik. A feltett kérdés szólhat mondjuk úgy, hogy melyik lényegi rész hasonlít igazán a felhasználó által feltett kérdésre. Ezek a hasonlóságok megfogalmazhatók egy speciális metrikával, amit a felhasználó kérdésére vett relevanciával össze lehet egyeztetni, például a következőképp:

Képzeljük el, hogy van egy d-dimenziós valós vektorterünk, $\mathbb{R}^d$, ahol most d elég nagy. Szeretnénk a kérdést és a lehetséges válaszokat elhelyezni ebben a térben valahogy úgy, hogy megadható legyen egy értelmes távolságmetrika a pontok között (mert a távolságnak szeretnénk megfeleltetni a relevanciát). Az OpenAI modellje egy $d = 1536$ dimenziós valós vektorteret használ. Ebben a konstrukcióban az egységgömb felszínére vetítjük a pontokat (koordinátákként raktározzuk el). Mivel távolságot akarunk definiálni, vehetjük a skaláris szorzatot, ebben az esetben az csak a két vektor bezárt szögétől függ, ami egyértelműen meghatároz egy gömbi távolságot.
Legyen tehát az u, v vektorok közötti távolság

$$d(u, v) = \langle u; v \rangle = cos(\theta)$$

módon definiálva, ahol θ az u és v vektorok közötti, origóból mért szög.
A válaszok és kérdések egységgömbre vetítése mögötti elgondolás hogy az egymáshoz hasonló dolgokat húzzuk közel egymáshoz a gömbön úgy, hogy közben tartsuk meg egyenletesen a többi választ is rajta (hogy ne történjen összehúzódás). Ezt embedding modelnek hívják. Egy ilyen lehetséges elrendezést taglal például a [WI20] cikk.

Ezen megfontolásokkal megadható a kérdés gömbfelszíni vetületéhez három legközelebb eső válasz vetülete, ezeket fogjuk meghatározni (ChromaDB mint könyvtáros), a választ ezek alapján formálja majd meg a nyelvi modell (LLM mint kutató). Az egész folyamatot RAG-nak nevezzük (Retrieval Augmentation Generation):

$$\text{RAG} : \text{Embedding model} \rightarrow \text{ChromaDB} \rightarrow \text{LLM}$$

1.2 Implementáció

Elsőnek létrehozunk egy virtuális környezetet, abban letöltjük a langchain, langchain-openai, labgchain-community, chromadb, pydf, python-dotenv könyvtárakat. Ezek egy szebb keretet adnak az előbbi megfontolások gyakorlatba ültetéséhez. A jelenlegi projekt egy FastAPI endpointos megoldást is ad tetszőleges PDF-ben való böngészésre, továbbá gpt 4.0 mini tokeneket használ az LLM rész megvalósításához.

Bibliography

[WI20] Tongzhou Wang and Phillip Isola. Understanding contrastive representation learning through alignment and uniformity on the hypersphere. *CoRR*, abs/2005.10242, 2020.